

OXVERSE ACADEMY

AI Vibe Coding Curriculum

AI Track (2 of 3) · Cohort-based · Project-driven

DURATION

2 Months (8 Weeks)

TOTAL WEEKS

8

LEVEL

Beginner to Job Ready

PROJECTS

10+

CAPSTONE

1 Launched AI-Built SaaS Product

PROGRAMME GOAL

Train complete beginners and career switchers to design, build, debug, and ship production-grade software products using AI-assisted development tools such as Lovable, Cursor, Claude Code, GitHub Copilot, v0, Bolt, and Replit, without needing years of traditional programming training.

OVERVIEW

AI Vibe Coding is an intensive 8-week, project-first program that teaches students how to build real software using AI as a full-time coding partner. Students learn just enough web fundamentals (HTML, CSS, JavaScript, React, TypeScript) to read and reason about AI-generated code, then rapidly layer on prompt-driven development, spec writing, PRD creation, Git/GitHub workflows, Supabase backends (auth, database, RLS, edge functions, storage), third-party API and payment integrations, debugging, testing, and deployment. By the end of the program every student will have shipped a real, live, monetizable SaaS product built primarily with AI tools, and will be equipped to freelance or join a team as an AI-native builder.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 913 869 1147

Course Outline

1. **Week 1:** Foundations of AI-Assisted Development & Web Literacy
2. **Week 2:** React, TypeScript & Component Thinking
3. **Week 3:** Prompt Engineering, Specs & PRDs for Builders
4. **Week 4:** Git, GitHub & Collaborative AI Workflows
5. **Week 5:** Supabase Backends: Auth, Database, RLS & Storage
6. **Week 6:** APIs, Payments & Third-Party Integrations
7. **Week 7:** Testing, Debugging & Deployment
8. **Week 8:** Capstone Launch, Monetization & Freelancing with AI

Foundations of AI-Assisted Development & Web Literacy

Students get oriented in the AI coding landscape and build just enough fundamental web literacy (HTML, CSS, JS basics) to read and evaluate what AI tools generate for them.

LEARNING OBJECTIVES

- Understand the AI vibe coding landscape and how tools like Lovable, Cursor, Claude Code, Copilot, v0, Bolt, and Replit differ
- Set up a complete AI-assisted development environment on a personal machine
- Read and explain basic HTML, CSS, and JavaScript code with confidence
- Write effective first prompts to scaffold a simple web page using an AI tool
- Understand the mental model of 'vibe coding' versus traditional hand-coding

DETAILED TOPIC BREAKDOWN

1.1 The AI Coding Tool Landscape

- What is vibe coding and why it matters in 2026
- Overview of Lovable: full-stack app builder with Supabase integration
- Overview of Cursor: AI-native code editor built on VS Code
- Overview of Claude Code: terminal-based agentic coding assistant
- Overview of GitHub Copilot: inline AI pair programmer
- Overview of v0 by Vercel: UI generation from prompts
- Overview of Bolt.new: in-browser full-stack scaffolding
- Overview of Replit: cloud IDE with AI agent and instant hosting
- Choosing the right tool for the right job
- Understanding LLM context windows and token limits
- How AI code generation actually works under the hood
- Limitations and hallucination risks of AI-generated code

1.2 Development Environment Setup

- Installing and configuring VS Code and Cursor
- Creating accounts: Lovable, Cursor, Claude, GitHub, Vercel, Supabase
- Installing Node.js, npm, and command line basics
- Navigating the terminal: cd, ls, mkdir, rm, pwd
- Understanding package managers: npm vs bun vs pnpm
- Setting up a GitHub account and SSH keys
- Browser DevTools tour: Elements, Console, Network tabs
- Installing helpful VS Code / Cursor extensions
- Configuring API keys and environment variables safely
- Understanding localhost, ports, and dev servers

1.3 HTML Fundamentals for Reading AI Code

- HTML document structure: doctype, head, body
- Common tags: div, span, section, header, footer, nav
- Semantic HTML and why AI tools favor it
- Forms: input, textarea, select, button, label
- Attributes: id, class, href, src, alt
- Lists, tables, and images
- Nesting and the DOM tree concept
- Reading generated HTML from an AI prompt and identifying structure
- Accessibility basics: alt text, aria labels, semantic landmarks

1.4 CSS Fundamentals and Modern Styling

Selectors, specificity, and the cascade Box model: margin, border, padding, content Flexbox layout fundamentals

CSS Grid layout fundamentals Responsive design with media queries Introduction to Tailwind CSS utility classes

Reading Tailwind class names generated by AI tools Color systems, spacing scales, and design tokens

Basic animations and transitions Dark mode implementation patterns

1.5 JavaScript Essentials

Variables: let, const, var and scope Data types: strings, numbers, booleans, arrays, objects

Functions, arrow functions, and callbacks Conditionals and loops Array methods: map, filter, reduce, forEach

Async JavaScript: promises and async/await The fetch API and making network requests DOM manipulation basics

Reading and tracing through AI-generated JS logic Common JavaScript errors and how to interpret them

1.6 Your First AI-Built Page

Writing a clear one-paragraph prompt describing a landing page Iterating on AI output through follow-up prompts

Identifying what changed in the code after each prompt Previewing changes live in Lovable/Bolt/Replit

Basic prompt hygiene: being specific about layout and content Exporting and viewing generated code in an editor

Comparing outputs across two different AI tools for the same prompt

HANDS-ON EXERCISES

- Install and configure Cursor, GitHub, and Supabase accounts
- Manually write a static HTML/CSS profile card without AI assistance
- Use Lovable to generate a landing page from a one-sentence prompt
- Trace through 20 lines of AI-generated JavaScript and annotate what each line does
- Recreate the same page in v0 and Bolt and compare the outputs

ASSIGNMENTS

- Build a personal 'about me' static page by hand, then rebuild it using an AI tool and write a short comparison of the experience
- Submit a written glossary explaining 15 HTML/CSS/JS terms in your own words

PROJECTS

- Personal landing page built with Lovable including hero section, about section, and contact form UI

WEEKLY LEARNING OUTCOMES

- Can navigate the terminal and basic dev environment confidently
- Can read and interpret basic HTML, CSS, and JavaScript code
- Understands the strengths and weaknesses of five major AI coding tools
- Has produced a first working AI-generated web page

WEEKLY ASSESSMENT

Practical quiz on HTML/CSS/JS reading comprehension plus a live demo of a working AI-generated landing page

React, TypeScript & Component Thinking

Students build enough React and TypeScript literacy to understand, modify, and extend the component-based codebases that AI tools generate by default.

LEARNING OBJECTIVES

- Understand component-based architecture and why AI tools default to React
- Read and modify JSX/TSX code confidently
- Understand TypeScript types, interfaces, and props
- Use state and props to build interactive AI-generated components
- Debug simple React rendering issues

DETAILED TOPIC BREAKDOWN

2.1 React Fundamentals

What is React and the virtual DOM concept JSX syntax and how it maps to HTML Components as functions returning UI

Props: passing data into components State with useState and re-rendering Conditional rendering patterns

Rendering lists with map and the key prop Event handling: onClick, onChange, onSubmit

Component composition and children props Lifting state up between sibling components

2.2 TypeScript for Vibe Coders

Why AI tools generate TypeScript by default Basic types: string, number, boolean, array, object Interfaces and type aliases

Typing component props Optional properties and union types Generics at a conceptual level

Reading and fixing common TypeScript errors reported by the editor Type inference vs explicit typing

Using TypeScript errors as a debugging signal when AI code breaks

2.3 React Hooks in Practice

useState deep dive with multiple state variables useEffect for side effects and data fetching

Dependency arrays and common pitfalls useRef for DOM references Custom hooks concept and simple examples

Context API for global state basics Recognizing hook misuse in AI-generated code

Common infinite loop bugs and how to spot them

2.4 Project Structure & Tooling

Anatomy of a Vite + React + TypeScript project Folder conventions: components, pages, hooks, lib, assets

Understanding package.json and dependencies Import/export syntax and module resolution

Using shadcn/ui and Tailwind together Environment variables in Vite projects

Reading a project's README and file tree generated by AI Navigating a large AI-generated codebase without getting lost

2.5 Modifying AI-Generated Components

Locating the right component file from a live preview

Making small text and style edits directly in code

Adding new props to an existing component safely

Extracting reusable components from duplicated code

Using AI chat inside Cursor/Lovable to request targeted edits

Verifying changes did not break other parts of the app

Using git diff to review exactly what an AI agent changed

HANDS-ON EXERCISES

- Build a counter component using `useState` from scratch
- Convert a plain JavaScript component into typed TypeScript
- Fix five intentionally broken React components with TypeScript errors
- Use Cursor's AI chat to add a new prop to an existing card component
- Trace an infinite re-render bug in a sample `useEffect` and fix it

ASSIGNMENTS

- Build a multi-component to-do list app (add, complete, delete) using React state, without a backend
- Take an AI-generated dashboard template and modify three components: add a new field, style change, and a new button with a handler

PROJECTS

- Interactive multi-page portfolio site built in React/TypeScript with reusable components (Navbar, Card, Footer, ContactForm)

WEEKLY LEARNING OUTCOMES

- Comfortable reading and editing JSX/TSX and TypeScript code
- Can use React state and props to build interactive UI
- Can safely modify AI-generated components without breaking the app
- Understands standard React + Vite project structure

WEEKLY ASSESSMENT

Code review exercise where the student must identify and fix three planted bugs in a React/TypeScript component

Prompt Engineering, Specs & PRDs for Builders

Students learn how to think and communicate like a technical product manager so their AI prompts produce production-quality results the first time.

LEARNING OBJECTIVES

- Write clear, structured prompts that consistently produce usable code
- Draft a professional Product Requirements Document (PRD) for a software feature
- Break a product idea into an actionable technical spec
- Use iterative prompting strategies to refine AI output
- Understand context management across a long AI coding session

DETAILED TOPIC BREAKDOWN

3.1 Prompt Engineering Foundations

- Anatomy of a high-quality coding prompt
- Specifying context, constraints, and desired outcome
- Role-based prompting: 'act as a senior React engineer'
- Few-shot prompting with code examples
- Iterative refinement: small diffs vs large rewrites
- When to ask for a plan before code
- Using negative constraints ('do not change X')
- Prompting for explanations alongside code
- Common prompt failure patterns and how to avoid them
- Chaining prompts across multiple features

3.2 Writing Product Requirements Documents

- Purpose and structure of a PRD
- Defining problem statement and target user
- Writing user stories and acceptance criteria
- Prioritization: must-have vs nice-to-have (MoSCoW)
- Defining scope boundaries and non-goals
- Success metrics and how to measure them
- Including wireframe or flow descriptions in a PRD
- Using a PRD as direct input for AI prompts
- Version history and iterating a PRD as a product evolves

3.3 Technical Spec Writing

- Breaking a feature into frontend, backend, and data requirements
- Defining data models before writing code
- Describing API endpoints and expected payloads
- Sketching user flows and edge cases
- Identifying third-party services needed (auth, payments, storage)
- Writing acceptance criteria testable by AI-generated tests
- Translating a spec into a sequence of AI prompts
- Documenting assumptions and open questions

3.4 Managing Long AI Coding Sessions

- Understanding context window limits in practice
- Structuring a project so AI agents have the right context
- Using README and architecture docs as persistent context
- Summarizing progress mid-session to reset context effectively
- Splitting large features into small independently testable prompts
- Recognizing when an AI agent has 'lost the plot' and needs a reset
- Using checklists to track multi-step AI-driven builds

3.5 Evaluating and Reviewing AI Output

- Reading a diff before accepting AI changes
- Checking for security issues in generated code (exposed keys, no validation)
- Verifying business logic matches the original spec
- Testing edge cases manually after AI generates a feature
- Requesting a second AI opinion or code review pass
- Recognizing over-engineered or unnecessarily complex AI output
- Establishing a personal pre-merge checklist

HANDS-ON EXERCISES

- Write three versions of a prompt for the same feature, from vague to highly specific, and compare outputs
- Draft a full PRD for a 'habit tracker' app feature
- Write a technical spec translating a PRD into data models and API endpoints
- Review an AI-generated pull request diff and list three issues found
- Practice mid-session context reset on a multi-step build

ASSIGNMENTS

- Produce a complete PRD and technical spec for your own SaaS idea (this will become your capstone concept)
- Use your spec to prompt an AI tool through building the first three features and document the prompt-by-prompt process

PROJECTS

- PRD + technical spec + first working prototype for a chosen SaaS idea

WEEKLY LEARNING OUTCOMES

- Can write prompts that reliably produce production-quality output
- Can independently draft a professional PRD and technical spec
- Can review AI-generated code diffs critically before accepting them
- Has selected and scoped their capstone SaaS idea

WEEKLY ASSESSMENT

Submission and peer review of a PRD, technical spec, and first working prototype build log

Git, GitHub & Collaborative AI Workflows

Students master version control and collaborative workflows essential for working safely with AI coding agents on real codebases.

LEARNING OBJECTIVES

- Use Git confidently for version control of AI-generated projects
- Collaborate on GitHub using branches, pull requests, and code review
- Recover safely from AI agents making unwanted changes
- Connect Lovable/Cursor/Bolt projects to GitHub repositories
- Understand CI basics for automated checks

DETAILED TOPIC BREAKDOWN

4.1 Git Fundamentals

- What version control is and why it matters with AI agents
- git init, add, commit, status, log
- Understanding the staging area and working directory
- Writing meaningful commit messages
- Branching: create, switch, merge
- Resolving merge conflicts step by step
- git diff and reading changes line by line
- Undoing changes: checkout, revert, reset
- Stashing work in progress
- Using .gitignore correctly for secrets and build files

4.2 GitHub Collaboration Workflows

- Creating and configuring a GitHub repository
- Cloning, pushing, and pulling
- Forking vs branching workflows
- Opening and reviewing pull requests
- Writing helpful PR descriptions
- Requesting and giving code review feedback
- Protected branches and required checks
- GitHub Issues for tracking bugs and features
- GitHub Projects boards for task management
- Connecting a repo to Vercel/Netlify for auto-deploy

4.3 Version Control with AI Agents

- Connecting Lovable to a GitHub repository
- Syncing Cursor/Claude Code changes to git commits
- Reviewing every AI agent commit before pushing to main
- Using feature branches for risky AI-driven refactors
- Rolling back a bad AI-generated change safely
- Preventing AI agents from committing secrets or API keys
- Setting up commit conventions for solo + AI-paired work
- Auditing AI agent commit history for accountability

4.4 Basic CI/CD Awareness

- What continuous integration is at a conceptual level
- Setting up a simple GitHub Actions workflow for linting
- Automated preview deployments on pull requests
- Reading CI failure logs and fixing them
- Why automated checks matter more with AI-generated code
- Adding a basic test-run step to CI

HANDS-ON EXERCISES

- Initialize a repo, make five commits with meaningful messages, and push to GitHub
- Simulate and resolve a merge conflict between two branches
- Connect a Lovable project to GitHub and inspect the generated commit history
- Roll back an intentionally broken AI-generated commit using git revert

- Set up a basic GitHub Actions lint workflow

ASSIGNMENTS

- Take your capstone project repo and implement a full branch-PR-review-merge cycle for one new feature
- Write a short 'incident report' simulating recovery from an AI agent accidentally deleting a working feature

WEEKLY LEARNING OUTCOMES

- Confident using Git for daily version control
- Can manage GitHub collaboration workflows independently
- Can safely recover from unwanted AI agent changes
- Has a working CI pipeline on their capstone project

WEEKLY ASSESSMENT

Live demonstration of creating a branch, making an AI-assisted change, opening a PR, and merging safely

PROJECTS

- Fully version-controlled capstone repository with branch protection, PR workflow, and CI lint check

Supabase Backends: Auth, Database, RLS & Storage

Students learn to design and implement full backend systems using Supabase, the default backend for most AI app builders.

LEARNING OBJECTIVES

- Design a relational database schema for a SaaS product
- Implement authentication flows using Supabase Auth
- Write and understand Row Level Security policies
- Use Supabase Storage for file uploads
- Build and deploy Supabase Edge Functions

DETAILED TOPIC BREAKDOWN

5.1 Database Design Fundamentals

- Relational database concepts: tables, rows, columns, keys
- Primary keys, foreign keys, and relationships
- One-to-many and many-to-many relationships
- Designing a schema from a PRD's data requirements
- Choosing appropriate column types
- Normalization basics and when to denormalize
- Using Supabase's table editor and SQL editor
- Writing basic SQL: SELECT, INSERT, UPDATE, DELETE, JOIN
- Migrations and schema versioning concepts

5.2 Supabase Authentication

- Email/password authentication setup
- Magic link and OTP authentication
- OAuth providers: Google, GitHub sign-in
- Session management and auth state in React
- Protecting routes based on auth state
- User profile tables linked to auth.users
- Password reset and email verification flows
- Role-based access patterns (admin vs regular user)
- Handling auth errors gracefully in the UI

5.3 Row Level Security (RLS)

- What RLS is and why it is critical for security
- Enabling RLS on a table
- Writing SELECT policies scoped to auth.uid()
- Writing INSERT, UPDATE, DELETE policies
- Testing policies using the Supabase policy simulator
- Common RLS mistakes AI tools make and how to catch them
- Public vs private data patterns
- Auditing an app for missing or overly permissive policies
- Using security definer functions safely

5.4 Storage and File Handling

- Creating storage buckets and setting public/private access
- Uploading files from a React form
- Generating signed URLs for private file access
- Image optimization and resizing considerations
- Storage security policies mirroring RLS concepts
- Handling file size limits and validation
- Displaying uploaded images/files in the UI

5.5 Edge Functions and Server Logic

- What Supabase Edge Functions are and when to use them
- Writing a simple Deno-based edge function
- Calling edge functions from the frontend
- Using edge functions to call external APIs securely
- Storing and using secrets in edge functions
- Triggering edge functions from database webhooks
- Debugging edge function logs
- Handling CORS in edge functions

5.6 Realtime and Advanced Queries

- Supabase Realtime subscriptions for live data
- Using Supabase client filters, ordering, and pagination
- Writing database functions and triggers
- Full text search basics in Postgres
- Combining multiple tables with joins in queries
- Performance considerations: indexes basics

HANDS-ON EXERCISES

- Design and implement a schema for a multi-user notes app with RLS
- Build a full email/password + Google OAuth login flow
- Write RLS policies so users can only see and edit their own rows
- Implement file upload to Supabase Storage with a private bucket and signed URL display
- Write and deploy an edge function that calls an external weather API securely

ASSIGNMENTS

- Design and implement the complete backend (schema, auth, RLS, storage) for your capstone SaaS idea
- Write a security audit document listing every table in your capstone and its RLS policy status

PROJECTS

- Fully functioning multi-user backend for the capstone project with auth, database, RLS, storage, and at least one edge function

WEEKLY LEARNING OUTCOMES

- Can design a relational schema from product requirements
- Can implement secure authentication with Supabase
- Understands and can audit Row Level Security policies
- Can build and deploy Supabase Edge Functions confidently

WEEKLY ASSESSMENT

Security and architecture review of the student's capstone backend, including a live RLS policy test

APIs, Payments & Third-Party Integrations

Students learn to integrate external services, payment processing, and third-party APIs into AI-built applications.

LEARNING OBJECTIVES

- Integrate third-party REST APIs into an AI-built application
- Implement a complete payment flow using Stripe
- Manage API keys and secrets securely across environments
- Handle webhooks for asynchronous events
- Debug failed API integrations methodically

DETAILED TOPIC BREAKDOWN

6.1 REST API Integration Fundamentals

- Understanding REST principles: endpoints, methods, status codes
- Reading third-party API documentation effectively
- Making authenticated API requests with headers and tokens
- Handling JSON request and response payloads
- Rate limiting and how to respect API quotas
- Error handling and retry strategies
- Using Postman or similar tools to test APIs before coding
- Proxying API calls through edge functions to hide secret keys

6.2 Stripe Payments Integration

- Stripe account setup and test mode
- Understanding Stripe Products, Prices, and Checkout Sessions
- Implementing Stripe Checkout for one-time payments
- Implementing Stripe subscriptions for recurring billing
- Handling successful and cancelled payment redirects
- Setting up Stripe webhooks for payment confirmation
- Storing subscription status in the Supabase database
- Building a customer billing portal
- Testing payments with Stripe test cards
- Going live: switching from test to production keys safely

6.3 Common Third-Party Integrations

- Email sending with Resend or similar services
- SMS and notifications with Twilio-style APIs
- AI API integration: calling OpenAI/Anthropic APIs from an app
- Map and location APIs
- Analytics integration (PostHog, Plausible, Google Analytics)
- File and image processing APIs
- Authentication with third-party OAuth beyond Supabase defaults

6.4 Webhooks and Async Events

- What webhooks are and why they matter
- Setting up a webhook endpoint with an edge function
- Verifying webhook signatures for security
- Idempotency and handling duplicate webhook events
- Logging and monitoring webhook activity
- Testing webhooks locally with tunneling tools

6.5 Debugging Integration Failures

Reading network tab requests and responses for failed calls

Distinguishing client errors (4xx) from server errors (5xx)

Common CORS errors and how to resolve them

Debugging environment variable misconfiguration

Using console logging strategically in edge functions

Asking AI tools for targeted debugging help with error messages

Building a personal integration troubleshooting checklist

HANDS-ON EXERCISES

- Integrate a public weather or news API into a React app
- Set up Stripe test mode and implement a one-time checkout flow
- Implement a Stripe subscription flow with webhook-based status updates
- Debug three intentionally broken API integrations with CORS, auth, and payload errors
- Send a transactional email using an email API triggered from an edge function

ASSIGNMENTS

- Add complete payment functionality (one-time or subscription) to your capstone SaaS product
- Integrate at least one additional third-party API relevant to your capstone (AI, email, analytics, or maps)

PROJECTS

- Capstone SaaS with fully working Stripe payment flow and at least one external API integration

WEEKLY LEARNING OUTCOMES

- Can integrate any documented REST API into an AI-built app
- Can implement and test a complete Stripe payment flow
- Understands and can debug webhook-based async workflows
- Can methodically debug failed API integrations

WEEKLY ASSESSMENT

Live demo of a working payment flow from checkout to confirmed subscription status in the database

Testing, Debugging & Deployment

Students learn to systematically test AI-generated code, debug production issues, and deploy applications reliably with custom domains and SEO.

LEARNING OBJECTIVES

- Write and run automated tests for critical application logic
- Systematically debug AI-generated bugs using a repeatable process
- Deploy a production application with a custom domain
- Implement basic SEO for a shipped product
- Monitor a live application for errors and performance issues

DETAILED TOPIC BREAKDOWN

7.1 Testing AI-Generated Code

- Why testing matters more, not less, with AI-generated code
- Manual test plans: writing a checklist from a PRD
- Unit testing basics with Vitest
- Writing tests for utility functions and business logic
- Component testing with React Testing Library
- End-to-end testing concepts with Playwright
- Asking AI tools to generate tests for existing code
- Reviewing AI-generated tests for actual coverage quality
- Setting up tests to run automatically in CI

7.2 Systematic Debugging

- The scientific method applied to debugging: hypothesis, test, verify
- Reproducing a bug reliably before attempting a fix
- Reading stack traces and error messages methodically
- Using browser DevTools breakpoints and the debugger statement
- Binary search debugging in large AI-generated files
- Isolating frontend vs backend vs database issues
- Writing an effective bug report prompt for an AI agent
- Avoiding the 'AI fix loop' where each fix creates a new bug
- When to roll back vs when to keep debugging forward

7.3 Deployment Fundamentals

- Comparing deployment targets: Vercel, Netlify, Lovable hosting
- Environment variables in production vs development
- Build process: understanding npm run build output
- Preview deployments vs production deployments
- Rolling back a bad deployment quickly
- Setting up staging environments for safer testing
- Zero-downtime deployment concepts

7.4 Custom Domains and DNS

- Buying a domain name and choosing a registrar
- Understanding DNS records: A, CNAME, TXT, MX
- Connecting a custom domain to a deployment platform
- Setting up SSL/HTTPS automatically
- Configuring subdomains for staging or marketing pages
- Setting up a custom domain email (e.g. hello@yourdomain.com)
- Troubleshooting DNS propagation issues

7.5 SEO and Discoverability

On-page SEO basics: title tags, meta descriptions, headings

Open Graph tags for social media sharing previews

Generating a sitemap.xml and robots.txt

Structuring URLs and content for search engines

Basic Core Web Vitals and page speed considerations

Submitting a site to Google Search Console

Using AI tools to generate SEO-optimized landing page copy

7.6 Monitoring and Maintenance

Setting up basic error tracking (e.g. Sentry-style tooling)

Reading application logs in production

Setting up uptime monitoring alerts

Tracking key product metrics post-launch

Planning a maintenance and update cadence

Handling user-reported bugs with a triage process

HANDS-ON EXERCISES

- Write five unit tests for a utility function using Vitest
- Debug a deliberately broken production build and document the resolution process
- Deploy a sample project to Vercel with a custom domain and HTTPS
- Add complete SEO meta tags and Open Graph tags to a project
- Set up basic error monitoring and trigger a test error to confirm alerts work

ASSIGNMENTS

- Write a full manual and automated test plan for your capstone project's critical user flows
- Deploy your capstone to production with a custom domain, SSL, SEO tags, and monitoring configured

PROJECTS

- Capstone SaaS deployed live to production with custom domain, SEO, and monitoring in place

WEEKLY LEARNING OUTCOMES

- Can write and run automated tests for critical app logic
- Follows a systematic, repeatable debugging process
- Can deploy and manage a production app with a custom domain
- Has implemented SEO and monitoring for a live product

WEEKLY ASSESSMENT

Live production deployment review including test coverage report, SEO audit, and monitoring dashboard walkthrough

Capstone Launch, Monetization & Freelancing with AI

Students finalize, polish, and publicly launch their capstone SaaS product, learn monetization and pricing strategy, and prepare to freelance or job-hunt as AI-native builders.

LEARNING OBJECTIVES

- Finalize and polish a capstone SaaS product to launch quality
- Define a pricing and monetization strategy for a real product
- Execute a public product launch across at least one channel
- Build a portfolio and profile positioned for AI-assisted development freelancing or employment
- Present a completed project confidently to a technical and non-technical audience

DETAILED TOPIC BREAKDOWN

8.1 Capstone Polish Sprint

- Conducting a full UX pass: consistency, spacing, empty states, loading states
- Writing clear error messages and empty state copy
- Mobile responsiveness audit across breakpoints
- Performance pass: reducing bundle size and load time
- Accessibility pass: keyboard navigation and screen reader basics
- Final security review checklist (RLS, exposed keys, input validation)
- Cleaning up unused code and dependencies
- Final QA pass using the test plan from Week 7

8.2 Pricing and Monetization Strategy

- Common SaaS pricing models: freemium, subscription, usage-based, one-time
- Researching competitor pricing for positioning
- Structuring pricing tiers and feature gating
- Writing a pricing page that converts
- Setting up Stripe pricing tiers to match strategy
- Basic unit economics: cost per user vs revenue per user
- Planning a free trial or freemium onboarding funnel

8.3 Go-To-Market and Launch Execution

- Writing a launch announcement post
- Launching on Product Hunt: preparation and timing
- Sharing on relevant communities (Reddit, X/Twitter, LinkedIn, niche forums)
- Building a simple landing page optimized for conversion
- Setting up basic analytics to track launch traffic and signups
- Collecting and acting on first-user feedback
- Planning a 30-day post-launch iteration roadmap

8.4 Freelancing and Career Positioning with AI Tools

- Building a portfolio site showcasing AI-built projects
- Writing case studies that show prompt-to-product process
- Setting up profiles on Upwork, Contra, and similar freelance platforms
- Pricing freelance AI-development services
- Writing effective client proposals for AI-assisted builds
- Positioning yourself for roles like 'AI-native developer' or 'no-code/low-code engineer'
- Networking in AI builder communities and Discord/Slack groups
- Preparing for technical interviews about your AI-assisted workflow

8.5 Final Presentation and Demo Day

- Structuring a compelling product demo narrative
- Live demo best practices and handling technical hiccups gracefully
- Explaining architecture and key technical decisions clearly to non-technical judges
- Answering tough questions about scalability and security
- Presenting metrics and traction from the launch week
- Receiving and incorporating final feedback

HANDS-ON EXERCISES

- Run a full accessibility and mobile responsiveness audit on the capstone project
- Draft three pricing tier structures and choose one with written justification
- Write a Product Hunt launch post and a supporting social media announcement thread
- Build a one-page portfolio site summarizing the capstone build process
- Do a full mock demo presentation and collect peer feedback

ASSIGNMENTS

- Complete the full polish sprint checklist and submit before/after evidence
- Publicly launch the capstone product on at least one channel and submit launch metrics after 72 hours

PROJECTS

- Capstone SaaS product fully launched, priced, and publicly live with a completed portfolio case study

WEEKLY LEARNING OUTCOMES

- Has launched a real, live, monetizable AI-built SaaS product
- Can define and justify a pricing and monetization strategy
- Has executed a public product launch and gathered real user feedback
- Has a professional portfolio and freelance/job-ready positioning as an AI-native builder

WEEKLY ASSESSMENT

Final Demo Day presentation with live product walkthrough, launch metrics review, and portfolio evaluation by instructors and peers